

SYSTEM DESIGN DOCUMENT

The Intelligent Inventory Dashboard

Domain: Supply

Task: Build an Intelligent Inventory Dashboard to give dealership managers a real-time overview of their vehicle stock.

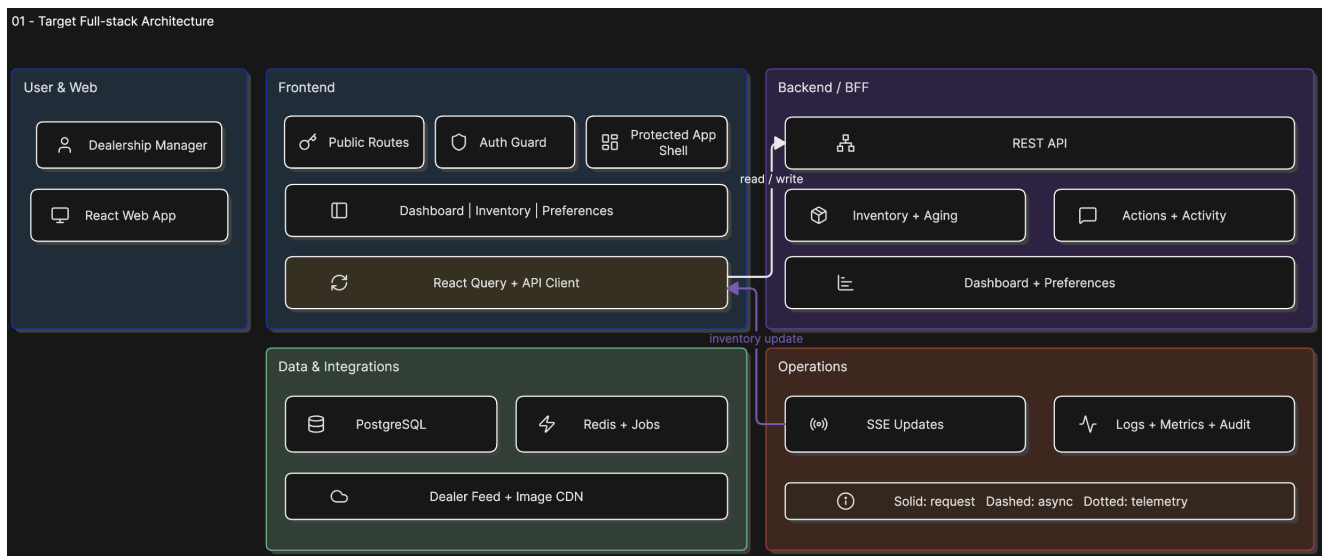
CORE REQUIREMENTS

1. Inventory Visualization: Display a filterable list of all vehicles in a dealership's inventory (e.g., filter by make, model, age).
2. Aging Stock Identification: Automatically identify and prominently display "aging stock" (vehicles in inventory for >90 days).
3. Actionable Insights: Allow a manager to log and persist a status or proposed action for each aging vehicle (e.g., "Price Reduction Planned").

Full content edition

Architecture Diagram

The Full-stack Solution



User & Web

We have here the user that mainly using this system are Dealership Managers and the selected platform was React Web App for the front-end structure

Front-end

This is the Front-end Structure

1. **Public Routes:** This one contain the pages of the authentication like login page and the error fallback pages
2. **Auth Guard:** This is a layout wrapper component to wrap the app shell make sure the user have the authority of these content (Have been logged in), its workflow:
 - Check the session state whether we have the token to access the data
 - Handle the pending state prevent the render error if the token verification was not return correctly
 - Reject if the token was not valid then force the user to redirect into the `/login`
 - Pass if the token was valid then render the children components that is the Protected App Shell
3. **Protected App Shell:** This is the layout that contain some universal component of the app like:
 1. App Navigation Bar: A side bar to navigate to the features Dashboard, Inventory, Preferences and some action like Logout
 2. Header bar: This is the section of breadcrumb
 3. Global Search Bar: For fast searching, navigate and some pre-defined actions
 4. Error Boundary: Have a error fallback component for the error situation like fall to catch the data of the inventory
 5. Main content Area: This is a dynamic display section for the features
4. **Features:**
 1. Dashboard: Show overview data with pre-set data table for quick view and statistic widget of the overall system like total items, aging items, import/export items
 2. Inventory: This will show the advance data table of the inventory
 3. Zone: This feature to show the zone definition like total slot, slot positions, priorities
 4. Preferences: This will show system setting like themes, Data grid density like compact or expand, data view default (Grid or Table)

5. Query & API Handler:

1. React Query setup for the global state of datas
2. API Client with Axios config for all the API calling

Backend / BFF

This is the main part of Backend contain the business logics

1. **RESTFul API**: Contain all the CURD with permission
2. **Inventory + Aging Logic**: Contain the logic of the inventory included items, position and zone, aging mark logic
3. **Actions and Activity Log**: Logging all the user behavior that affect to the data
4. **Dashboard Widget data support and preferences**: Some endpoint logic that support for the dashboard quick call endpoint like preset of the data table or statistic data for the dashboard widgets

Operations

This one is a module belong to the server-side ecosystem

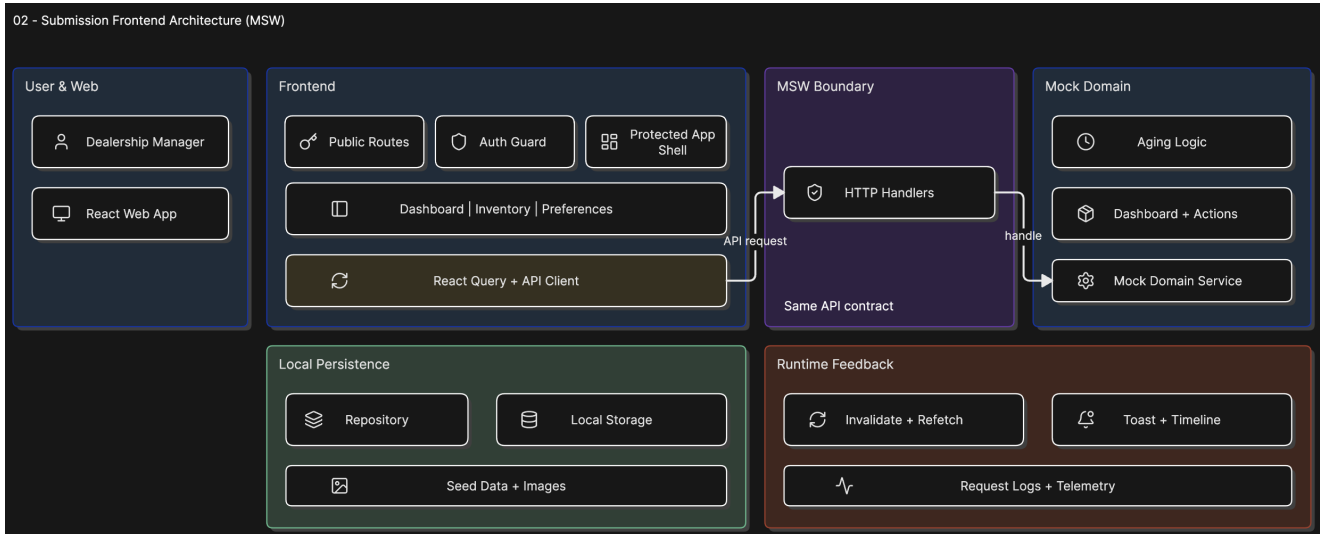
1. **SSE Updates**: This service contain the logic to push the data to the client side that connected, keep the API not overload by keeping the connection alive
2. **Logs + Metrics + Audit**: This is the observation services for Frontend app and Backend app

Data & Integrations

This contain the database and other services integrations

1. **PostgreSQL** : A SQL database that suit for multi scale business and related data like this scenarios
2. **Redis + Jobs**: For caching feature and cron jobs:
 1. Checking the outdated vehicle (over >90 days)
 2. Publishing message for the inventory alert to the SSE service that subscribing this pub/sub channel then broadcast to the current Client Side
 3. Asynchronous worker that handle a huge task like large import data or multi step task
3. **Dealer Feed and Image CDN**:
 1. Dealer Feed support for inbound and outbound data
 2. Image CDN for global image caching and delivering

Submission Solution



MSW Boundary and Mocking Domain

1. **HTTP Handlers:** MSW HTTP Handlers with the same API contract of the full-stack's backend solution act like a API Gateway
2. **Aging Logic :** Handle the aging mark logic
3. **Dashboard + Actions :** Dashboard statistic widget data and presets, action of the business logic
4. **Mock domain service :** Handle the connection to the persistence instance like local storage or files to perform actions also save the activity log

Runtime Feedback

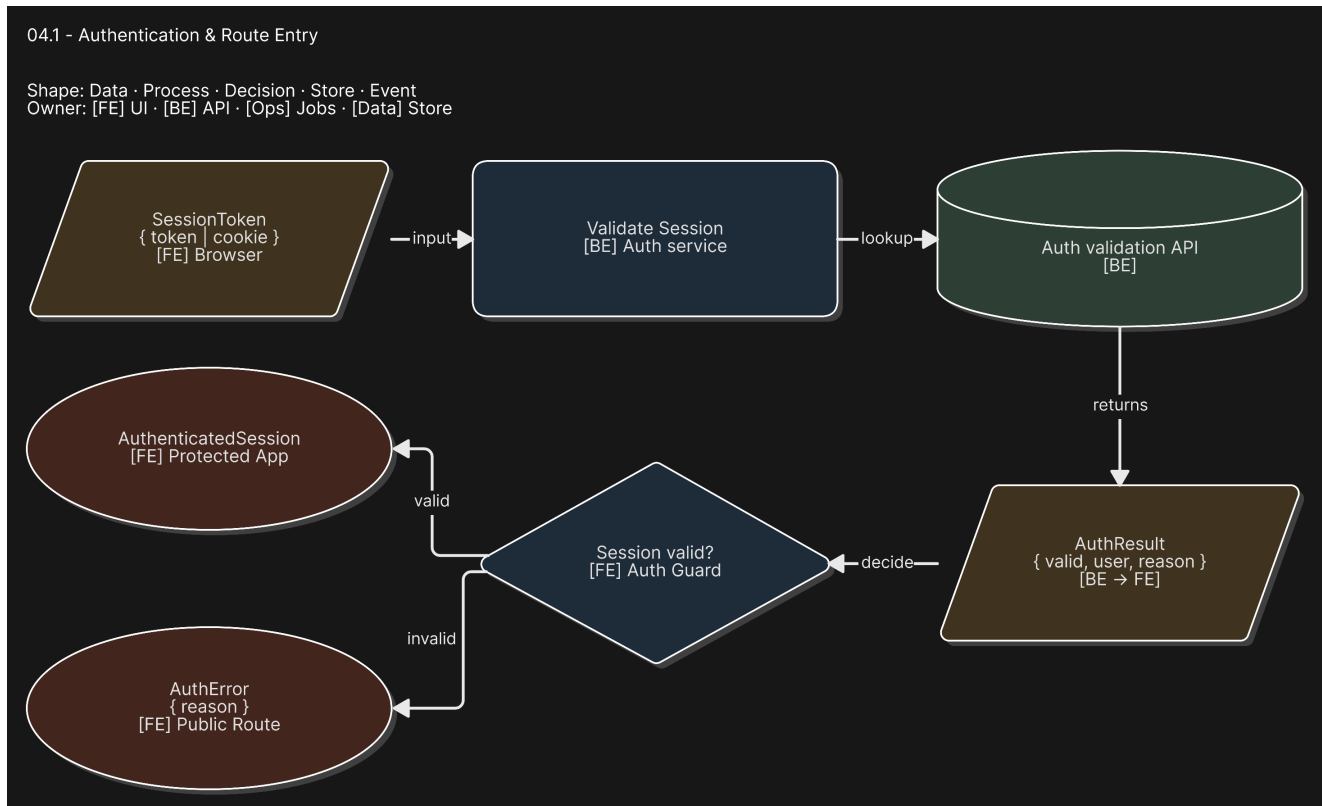
1. **Invalidate + Refetch :** Handle using the React Query hooks
2. **Toast + Timeline :** Setup toast and timeline (activity log)
3. **Request log + Telemetry :** Using the MSW life-cycle events to log the requests, stimulate the latency with delay function and using the performance benchmark to measure the request. Also have a error log tracking using Singleton Logger Utility

Local Persistence

1. **Repository:** Contain the whole codebase with clean rules
2. **Local Storage :** Main replicate database for the MSW and client persistence data
3. **Seed Data + Images :** Using AI that help to crawl some schema matching data and images

Dataflow explanation

1. Authentication & Route Entry



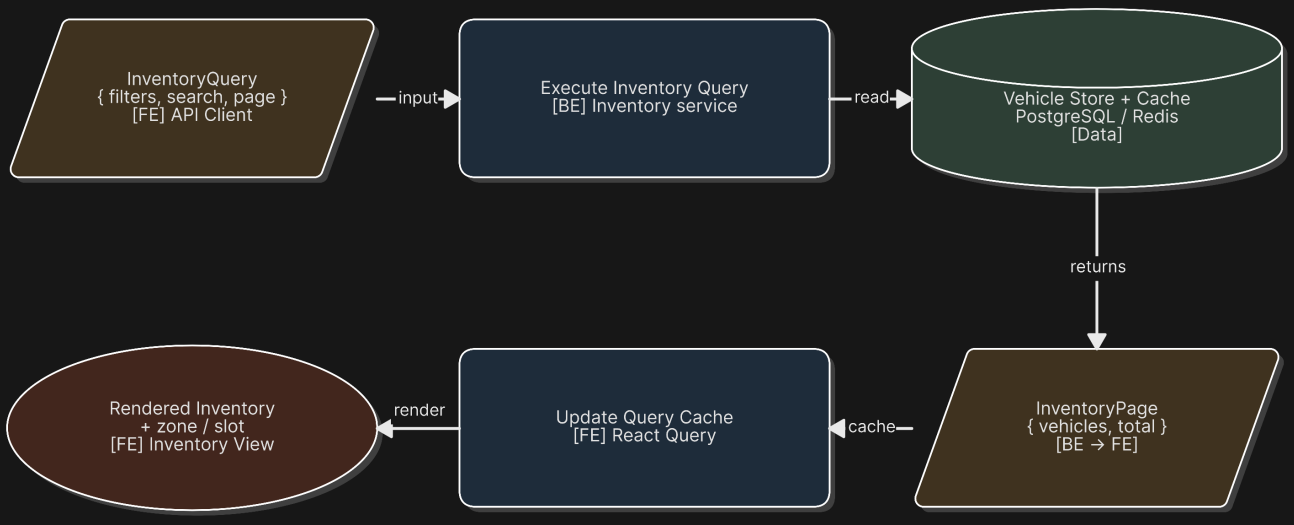
This flow happens when the user opens the app or accesses a protected page.

1. **Session Token:** Browser sends the current token or cookie from the user session.
2. **Validate Session:** Auth service checks this token with the auth validation API.
3. **Auth Result:** The API returns the user data when it is valid, or returns the reason when it is not valid.
4. **Auth Guard:** Front-end decides the route. Valid users can go to the Protected App, invalid users go back to the Public Route like </login>.

2. Inventory Browse & Search

04.2 - Inventory Browse & Search

Shape: Data · Process · Decision · Store · Event
Owner: [FE] UI · [BE] API · [Ops] Jobs · [Data] Store



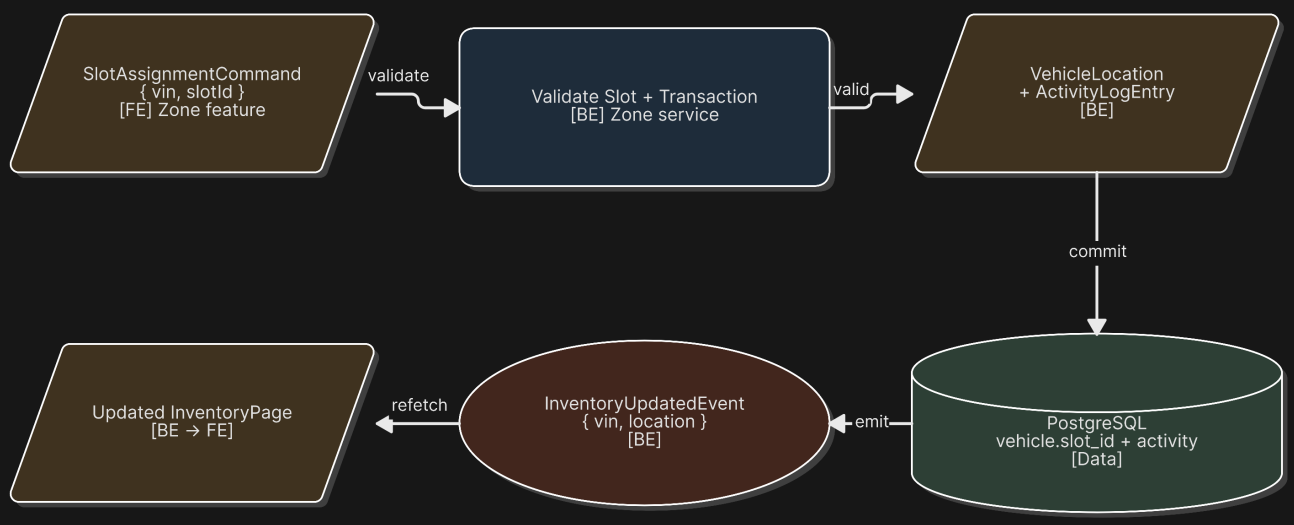
This flow used when manager open Inventory page, search a vehicle or change the filters.

1. **Inventory Query:** Front-end send filters, search keyword and current page to the Inventory service.
2. **Execute Inventory Query:** Backend read vehicle data from PostgreSQL, or use Redis cache when the data already available.
3. **Inventory Page:** Backend return the vehicles list and total result back to Front-end.
4. **React Query Cache:** Front-end save this result in query cache then render the inventory table with zone and slot information.

3. Zone & Slot Assignment

04.3 - Zone & Slot Assignment

Shape: Data · Process · Decision · Store · Event
Owner: [FE] UI · [BE] API · [Ops] Jobs · [Data] Store

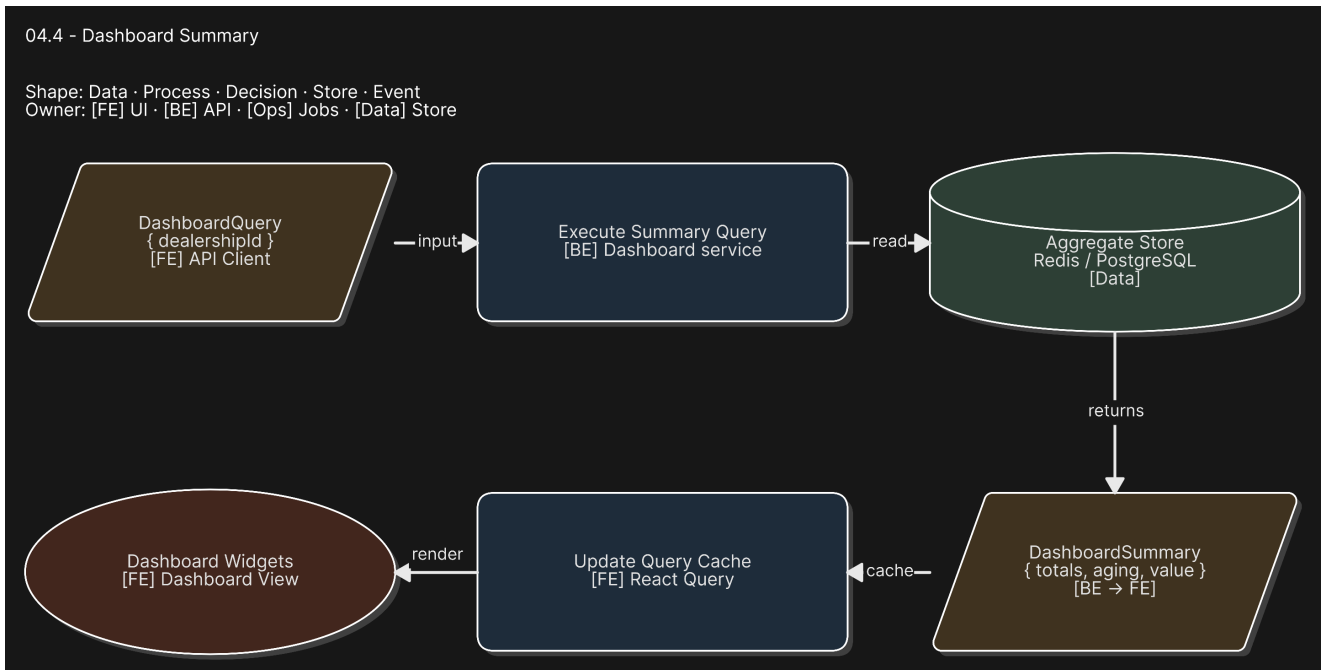


This flow used when manager assign or move a vehicle into a slot in the dealership lot.

1. **Slot Assignment Command:** Zone feature send vehicle VIN and selected slot id to the Zone service.

2. **Validate Slot + Transaction:** Backend make sure the slot is available, then update the vehicle location and create an activity log in one transaction.
3. **PostgreSQL:** The new `vehicle.slot_id` and activity data are persisted here.
4. **Inventory Updated Event:** Backend publish the changed location so the current inventory page can load the updated data.

4. Dashboard Summary



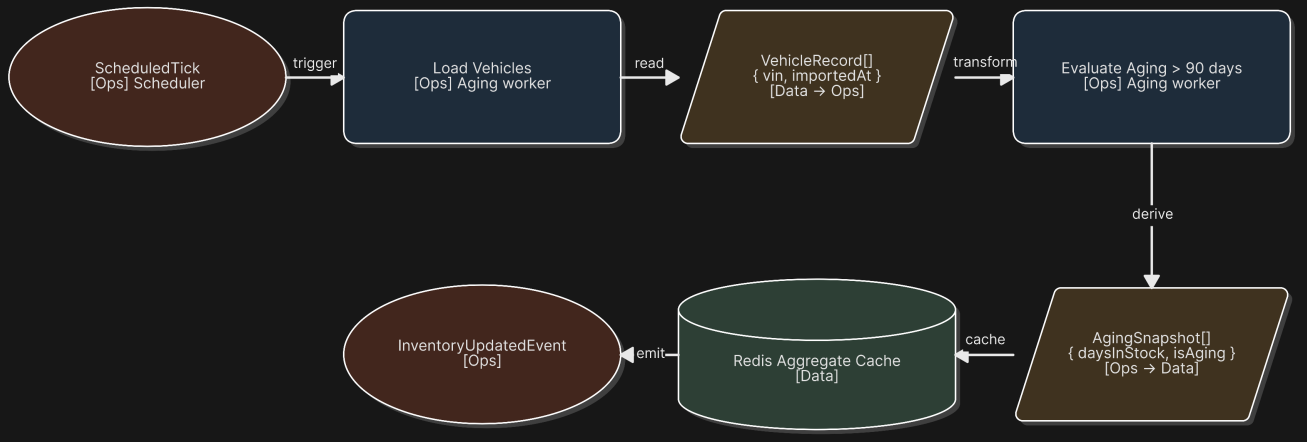
This flow used when manager open Dashboard or refresh the dashboard widgets.

1. **Dashboard Query:** Front-end ask for the summary of one dealership.
2. **Execute Summary Query:** Dashboard service read the prepared aggregate data from Redis or calculate it from PostgreSQL.
3. **Dashboard Summary:** Backend return totals, aging count and estimated value to the Front-end.
4. **React Query Cache:** Front-end save the summary then render the statistic widgets.

5. Aging-Stock Evaluation

04.5 - Aging-Stock Evaluation

Shape: Data · Process · Decision · Store · Event
Owner: [FE] UI · [BE] API · [Ops] Jobs · [Data] Store



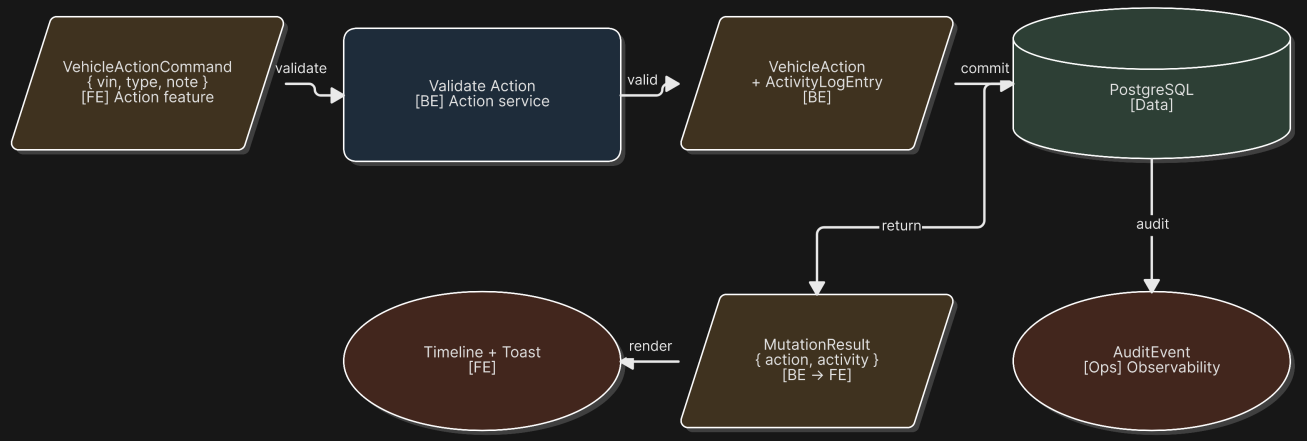
This flow is a background job. It make sure the system can identify the vehicles that stay too long in the inventory.

1. **Scheduled Tick:** Operations scheduler trigger the Aging worker by schedule.
2. **Load Vehicles:** Worker read vehicle VIN and imported date from the data store.
3. **Evaluate Aging:** Worker calculate **daysInStock**. A vehicle is aging when it is over 90 days.
4. **Aging Snapshot:** The calculated result is saved into Redis aggregate cache, then an inventory updated event is emitted for the client refresh flow.

6. Manager Action & Activity

04.6 - Manager Action & Activity

Shape: Data · Process · Decision · Store · Event
Owner: [FE] UI · [BE] API · [Ops] Jobs · [Data] Store



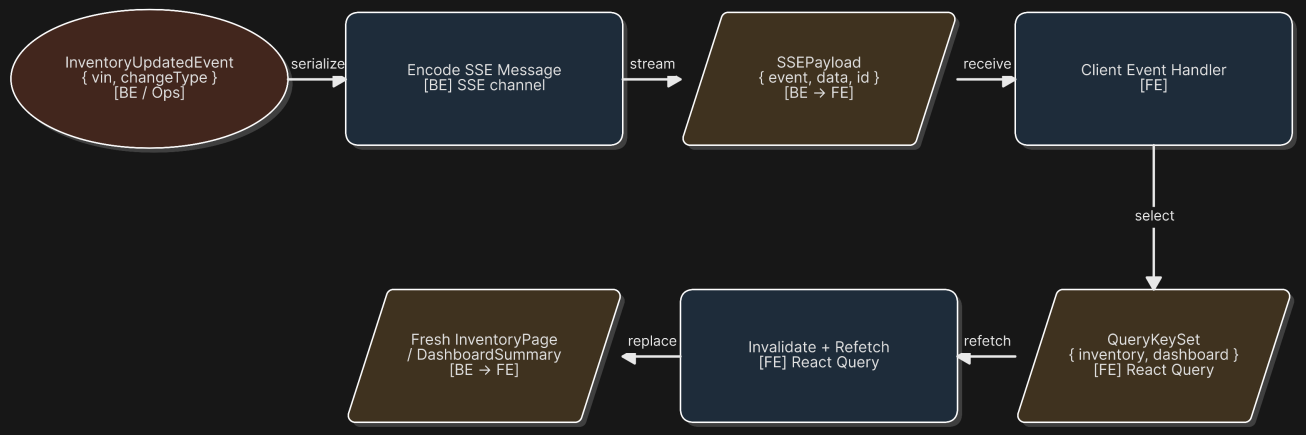
This flow used when manager add an action or a note for a vehicle, for example **Price Reduction Planned**.

1. **Vehicle Action Command:** Action feature send VIN, action type and note to the Action service.
2. **Validate Action:** Backend check this action then create both Vehicle Action and Activity Log Entry.
3. **PostgreSQL:** Action and activity are saved together so the system keep the history of the change.
4. **Result and Audit:** Backend return the new action to Front-end for Timeline and Toast. At the same time an Audit Event is sent to Operations for tracking.

7. SSE-driven UI Refresh

04.7 - SSE-driven UI Refresh

Shape: Data · Process · Decision · Store · Event
Owner: [FE] UI · [BE] API · [Ops] Jobs · [Data] Store



This flow keep the client data fresh after an inventory change without the user manually reload the page.

1. **Inventory Updated Event:** Backend or worker publish an event when inventory data was changed.
2. **SSE Message:** SSE channel convert the event into a small message and stream it to the connected Front-end.
3. **Client Event Handler:** Front-end receive the message then select the inventory and dashboard query keys that need to update.
4. **Invalidate + Refetch:** React Query clear the old cache and call the API again. The UI get fresh Inventory Page or Dashboard Summary from the source of truth.

Technologies decision

#	Options	Decision
Front-end		
1. Framework	Next.js (App Router) Vite + React Remix	Vite + React Fits a dashboard application: simple setup, clean architecture, and full control over client-side data fetching. SEO and SSR are not needed.
2. State Management	React Query + local state Zustand Redux Toolkit	React Query + local state Keeps the stack lightweight. Server state stays in React Query; small shared UI state can use Context + reducer when needed.
3. API Client	Native fetch wrapper Axios OpenAPI generated client	Axios Easy to configure now, with a clear path to interceptors and more advanced request handling later.
4. Routing	Next.js App Router React Router TanStack Router	TanStack Router Modern, type-safe routing with a simple file-based approach that suits the application structure.
5. Forms + Validation	React Hook Form + Zod Formik + Yup TanStack Form + Zod	TanStack Form + Zod Simple to set up, with clear form state and schema validation in one flow.
6. Styling	Tailwind CSS MUI CSS Modules	Tailwind CSS Strong ecosystem, clean utility-first styling, and fast composition directly in components.
Back-end		
1. Runtime + Framework	Node.js + NestJS Node.js + Fastify Bun + Elysia	Node.js + NestJS Provides a professional project structure and strong module boundaries, while leaving room to scale later.
2. Application Architecture	Modular monolith: Auth; Inventory + Zone + Aging; Actions + Activity; Dashboard + Preferences Layered monolith by technical concern Independent microservices by domain	Modular monolith Easy to follow in the early stage. A module can be extracted into a microservice later if it grows independently.
3. API Style + Contract	REST + OpenAPI GraphQL tRPC	REST + OpenAPI Simple to design and consume, with endpoints that are easy to inspect and debug.
4. Authentication + Authorization	Better Auth + RBAC Passport.js + JWT Auth0 + RBAC	Better Auth + RBAC Simple authentication and role-based access control that fits the current project scope.
Operations		
1. Real-time Client Updates	Server-Sent Events (SSE) WebSockets Polling	Server-Sent Events (SSE) Only one-way broadcast is needed: the server pushes inventory updates to subscribed clients. Two-way WebSocket communication would add unnecessary complexity.
2. Background Jobs + Scheduling	BullMQ + Redis node-cron Managed scheduler + worker	BullMQ + Redis A proven queue for scheduled aging scans and long-running imports. Redis-backed retries, delayed jobs, and worker separation keep API requests fast and reliable.
3. Observability + Audit	Sentry + Pino + OpenTelemetry + Grafana Sentry + structured logs Cloud-provider monitoring	Sentry + Pino + OpenTelemetry + Grafana Sentry tracks Front-end errors with source maps and breadcrumbs. Pino handles Backend logs, OpenTelemetry keeps tracing vendor-neutral, and Grafana monitors metrics.
Data		
1. Primary Database	PostgreSQL MySQL MongoDB	PostgreSQL Reliable for business logic and related data, with strong relational queries and dependable performance.
2. Cache + Queue Store	Redis KeyDB PostgreSQL only	Redis An industry-standard in-memory store for fast caching and queue-backed background jobs.
3. Vehicle Image Storage + Delivery	Cloudflare R2 + CDN Amazon S3 + CloudFront Cloudinary	Cloudflare R2 + CDN Cost-effective object storage with CDN delivery built in for vehicle images.

Observability Strategy

Full-stack solution

The strategy gives the team one connected view of errors, requests, background jobs, and infrastructure health. Each tool has one clear responsibility.

1. **Front-end errors — Sentry:** Capture JavaScript errors, failed API calls, and key user actions. Source maps make production errors readable, while breadcrumbs show the user path before the issue happened.
2. **Backend logs — Pino:** Write fast structured JSON logs for every REST request, domain action, job result, and unexpected error. Each log includes `requestId`, `traceId`, module, and log level so it can be searched and connected to a request.
3. **End-to-end tracing — OpenTelemetry:** Propagate a `traceId` from the browser through the API, PostgreSQL, Redis, BullMQ worker, and SSE broadcast. This makes slow or failed flows traceable without tying the system to one monitoring vendor.
4. **Metrics and dashboards — Grafana:** Monitor API latency and error rate, PostgreSQL/Redis health, queue depth and failed jobs, plus active SSE connections. Alert on sustained error spikes, slow requests, or failed aging scans.
5. **Audit is separate from observability:** Manager changes such as status, zone, slot, action plan, and notes are stored as durable activity/audit records. Logs and traces help diagnose a problem; audit records answer who changed business data and when.

Errors are always captured. Normal traffic can be sampled to control monitoring cost. Tokens, cookies, passwords, and unnecessary personal data must never be sent to Sentry, logs, or traces.

Submission Solution

The submission uses MSW observability to make the mock API behave like a real network boundary and to make every request easy to inspect during review.

1. **Metrics — simulated latency and execution time:** Every MSW handler waits for a random `200–1000ms` delay before returning data. This prevents unrealistic near-zero Local Storage responses and lets the UI show loading skeletons and pending states. Record the start and finish time with `performance.now()` through MSW lifecycle events, then log the simulated duration for each request.
2. **Logging — structured console groups:** Replace scattered `console.log` calls with `console.groupCollapsed`. Log a muted request line when a request starts; on success, open a green response group containing the correlation ID, method, URL, status code, headers, and response body. Errors use the same structure with error details instead.
3. **Tracing — simulated correlation ID:** The Axios API client creates a UUID for every request and sends it as `X-Correlation-ID`. MSW reads this header and includes the same ID in request, response, and error logs. One ID can then connect the client action, mock handler, latency measurement, and final response.

This is submission-only instrumentation. It demonstrates the production observability flow without presenting MSW console logs as a replacement for Sentry, Pino, OpenTelemetry, or Grafana.

GenAI-Assisted Design Process

Scope and collaboration

The design brief started with three outcomes I wanted to protect: a filterable inventory, a clear signal for vehicles held longer than 90 days, and an action that could be persisted for each aging vehicle. I owned the product priorities, selected the visual direction, provided targeted feedback, and made the final scope decision. Codex acted as the design partner: it explored alternative structures, produced responsive prototypes, defined architecture boundaries, and turned my feedback into review-ready iterations. The collaboration followed a short loop: **my brief** → **Codex options** → **visual prototype** → **my review** → **Codex refinement**.

Tools used

- Design exploration ran through **Codex and the Huashu Design workflow**, covering information architecture, interaction flows, edge cases, and handoff notes.
- Codex expressed its proposals as **versioned Huashu HTML/CSS prototypes**, so I could review concrete screens without confusing design evidence with production behavior.
- **Browser viewport captures and interaction checks** gave me a consistent way to compare desktop, tablet, and mobile layouts and catch hierarchy or overflow problems early.
- **AG Grid, Animate UI, Base UI, and Iconsax** served as implementation guardrails for Codex's proposals. They were reference points, not GenAI output or a theme to copy directly.

Version journey

The collaboration progressed through **five major versions and nine recorded review milestones**. The **final** page is only the curated review entry, not another design version.

Version	Design outcome for the core requirements
v0.1	Codex prepared three visual directions. I chose Precision Cockpit as the base, retained the calmer typography and spacing from Direction A, and carried forward the stronger status/overlay composition from Direction B. Codex consolidated those choices in v0.1-d.
v0.2	Codex translated the selected direction into the inventory data surface and reusable patterns for filters, actions, durable status, feedback, and decision overlays. My targeted feedback shaped v0.2.1; later iterations prepared the patterns for full-screen composition.
v0.3	Codex composed the core responsive flow: Overview → Inventory → Inventory Unit → Activity . The resulting contract kept detailed comparison on desktop/tablet and used task-specific cards and focused sheets on mobile.
v0.4	Codex explored broader management screens. I decided that location and zone/slot management diluted the three requested outcomes, so those screens remained future design evidence rather than core scope.
v0.5	I froze the scope around the three core outcomes. Codex then documented the implementation-ready handoff and ownership model: Inventory Unit owns VIN-specific age and latest proposed action; Activity owns timestamped evidence.

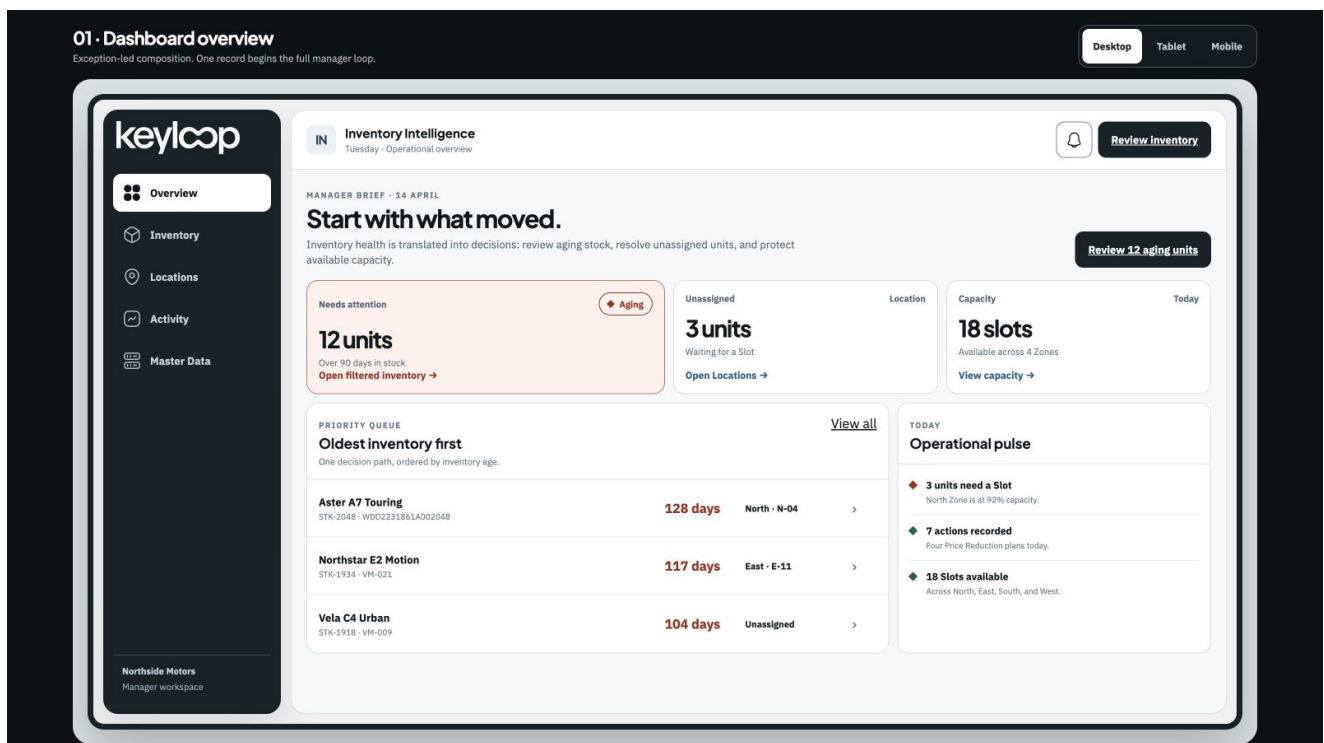
How the design adapted

The most important adaptation came from my scope decision after the broader exploration. Vehicle Master stayed as supporting data for make/model filtering, while unrelated management workflows were deferred. Codex converted that decision into a smaller core journey and made the original aging rule explicit: `daysInStock > 90` is aging; exactly 90 days is not.

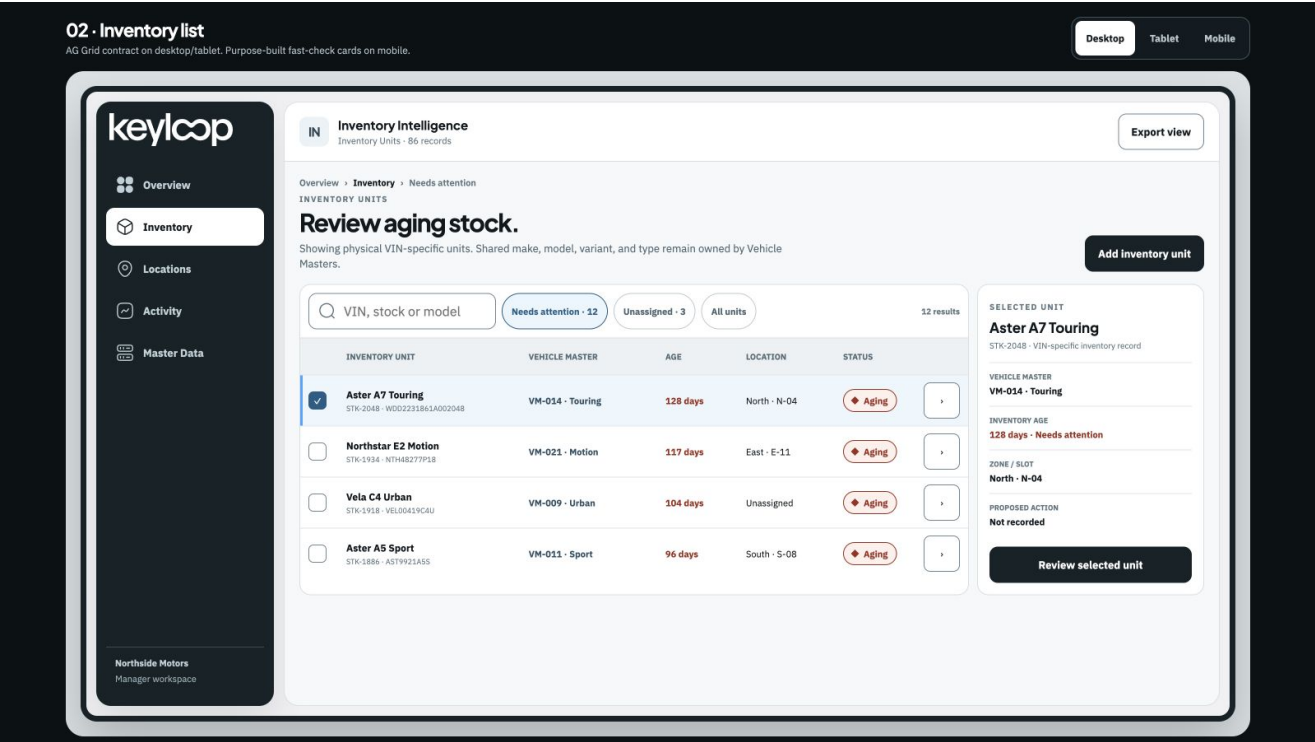
Codex also formalised the action flow as an architectural contract rather than a visual-only form: a successful save updates the vehicle's latest proposed action, appends a timestamped Activity entry, and refreshes the affected views. I approved the separation between screen design and persistence, allowing the same UI contract to work with an API and PostgreSQL in the full system or with MSW and Local Storage in the bounded submission.

Design evidence

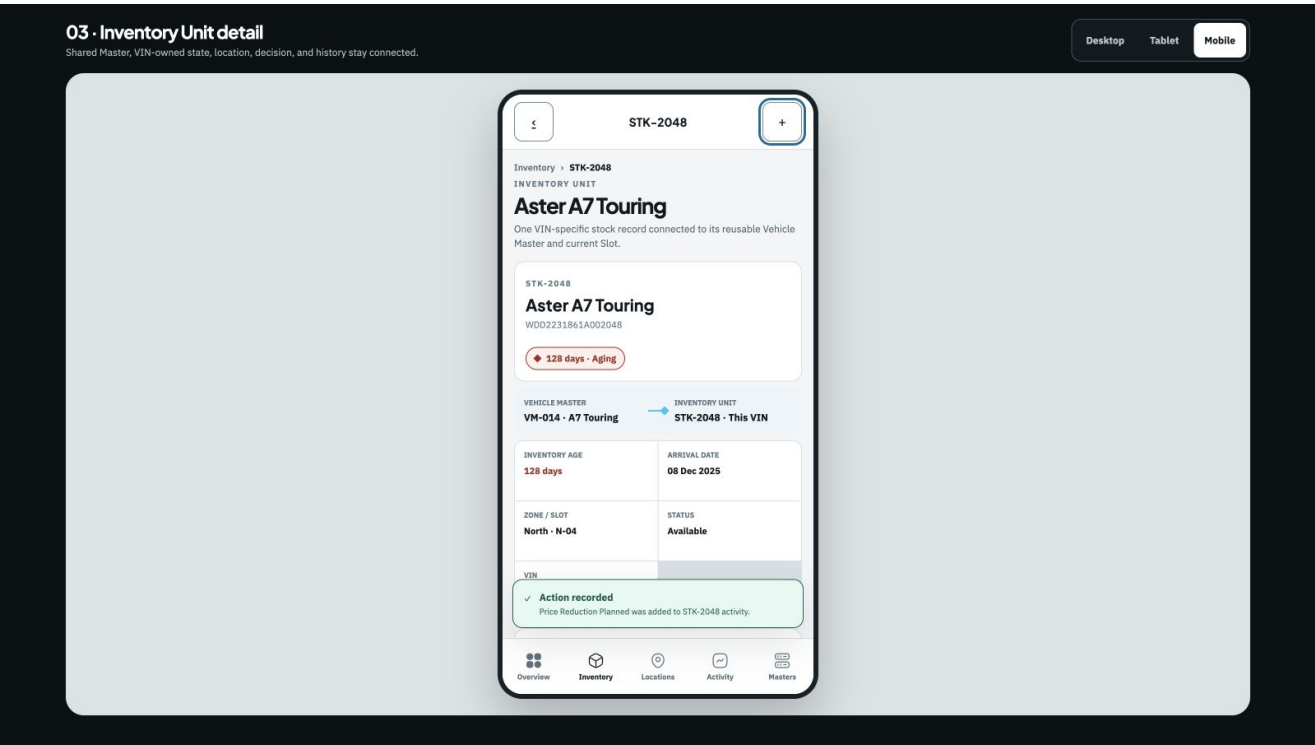
Dashboard overview — aging stock is elevated into the primary attention signal, with the oldest vehicles ordered first so the next decision is immediately visible.



Inventory and aging-stock signal — the list supports search/filtering and gives vehicles over 90 days a prominent, text-labelled treatment.



Proposed action feedback — the aging vehicle remains in context while the interface confirms that the action was recorded.



Persisted activity evidence — the action is visible with stock identity, timestamp, actor, and note.

08 · Activity history

The manager decision becomes durable, attributable operational context.

DesktopTabletMobile

keyloop

Overview

Inventory

Locations

Activity

Master Data

Northside Motors
Manager workspace

INInventory Intelligence

Activity · All operational events

Export activity

STK-2048 · Activity

OPERATIONAL HISTORY

Decisions stay visible.

Manager actions, location changes, and system events remain attached to the records they affected.

FIND ACTIVITY

Keep context narrow.

RECORD

STK-20483

All inventory28

EVENT TYPE

All events

Manager actions

Location changes

DATE

Last 7 days

Last 30 days

STK-2048 · 3 EVENTS

Aster A7 Touring

Action recorded

Today
14:32

Price Reduction Planned

Proposed action recorded by Krist Manager.

Review pricing against comparable A7 Touring units before Friday.

STK-2048 →

12 Apr
10:18

Moved to North · N-04

Inventory Unit assignment changed from East · E-02.

STK-2048 →

10 Apr
09:41

Inventory inspection completed

Unit marked Available and ready for manager review.

STK-2048 →